



UNIVERSIDADE
FEDERAL DO CEARÁ

LÓGICA PARA COMPUTAÇÃO

ALINE VITORIA SANTANA NUNES
CAROL LIMA FONSECA

Trabalho Final

Russas, CE
2026

Sumário

1. Descrição do problema.....	3
2. Representação dos grafos.....	3
3. Estratégia utilizada.....	3
4. Predicados implementados.....	3
5. Casos de teste.....	4
5.1 Teste positivo 1.....	4
5.2 Teste positivo 2.....	4
5.3 Teste negativo 1.....	5
5.4 Teste negativo 2.....	5
5.5 Teste negativo 3 (caso limite).....	5
5.5 Grafo 25.....	5
6. Capturas de tela das consultas.....	7
6.1 Print do teste_positivo_1.pl executando clique4.....	8
6.2 Print do teste_positivo_1.pl executando mostra_clique4.....	8
6.3 Print do teste_positivo_2.pl executando clique4.....	9
6.4 Print do teste_positivo_2.pl executando mostra_clique4.....	9
6.5 Print do teste_negativo_1.pl executando clique4.....	10
6.6 Print do teste_negativo_1.pl executando mostra_clique4.....	10
6.7 Print do teste_negativo_2.pl executando clique4.....	11
6.8 Print do teste_negativo_2.pl executando mostra_clique4.....	11
6.9 Print do teste_negativo_3.pl executando clique4 e mostra_clique4.....	12
6.10 Print do grafo 25 executando clique4.....	12
6.11 Print do grafo 25 executando mostra_clique4.....	12
7. Resultados obtidos.....	14
8. Conclusão.....	13

1. Descrição do problema

O trabalho tem como objetivo resolver o problema de detecção de clique de tamanho 4 em grafos simples, não direcionados e finitos, usando a linguagem lógica SWI-Prolog. Uma clique é um conjunto de vértices em que todos os pares de vértices distintos são adjacentes. Assim, uma clique de tamanho 4 é formada por quatro vértices A, B, C e D tais que existem arestas entre todos os pares:

- A e B
- A e C
- A e D
- B e C
- B e D
- C e D

O problema de decisão consiste em determinar se existe pelo menos um conjunto de quatro vértices que satisfaçam essa propriedade. O predicado principal esperado no trabalho é:

```
clique4.
```

2. Representação dos grafos

Os grafos foram representados por fatos do tipo `adj(X,Y)`, indicando que o vértice X é adjacente ao vértice Y. Como os grafos são não direcionados, cada aresta foi representada nas duas direções. Por exemplo, uma aresta entre a e b foi escrita como:

```
adj(a,b).  
adj(b,a).
```

Essa modelagem permite que o Prolog consulte a adjacência entre vértices independentemente da ordem em que eles aparecem.

3. Estratégia utilizada

A solução usa o mecanismo de busca e backtracking do Prolog. O predicado `clique4` tenta encontrar quatro vértices A, B, C e D que estejam conectados entre si dois a dois. Para isso, o predicado verifica explicitamente as seis adjacências necessárias para formar uma clique de tamanho 4:

```
adj(A, B), adj(A, C), adj(A, D),  
adj(B, C), adj(B, D),  
adj(C, D)
```

Se todas essas relações forem verdadeiras para alguma combinação de quatro vértices, então existe uma clique de tamanho 4 no grafo. A condição:

```
A @< B, B @< C, C @< D
```

foi usada para impor uma ordem entre os vértices. Isso evita que a mesma clique seja encontrada várias vezes em ordens diferentes, como {a,b,c,d}, {b,a,c,d} ou {d,c,b,a}. Quando uma tentativa falha, o Prolog realiza backtracking, escolhendo novas possibilidades para os vértices até encontrar uma clique ou até esgotar todas as combinações possíveis.

4. Predicados implementados

O predicado principal implementado foi `clique4`, responsável por responder se existe uma clique de tamanho 4:

UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE RUSSAS

```
clique4 :-  
    adj(A, B), adj(A, C), adj(A, D),  
    adj(B, C), adj(B, D),  
    adj(C, D),  
    A @< B, B @< C, C @< D.
```

Também foi implementado o predicado auxiliar `mostra_clique4`, usado para imprimir uma clique encontrada ou informar que nenhuma clique de tamanho 4 existe no grafo:

```
mostra_clique4 :-  
    adj(A, B), adj(A, C), adj(A, D),  
    adj(B, C), adj(B, D),  
    adj(C, D),  
    A @< B, B @< C, C @< D, !,  
    format("Clique de tamanho 4 encontrada: {~w, ~w, ~w, ~w}~n", [A,B,C,D]).  
mostra_clique4 :-  
    write('Nenhuma clique de tamanho 4 encontrada.'), nl.
```

O operador de corte `!` foi usado no predicado `mostra_clique4` para interromper a busca depois que a primeira clique é encontrada. Dessa forma, a saída mostra apenas uma clique, em vez de listar todas as combinações possíveis.

5. Casos de teste

Foram usados quatro grafos pequenos, com 5 a 8 vértices, além do grafo grande escolhido pela equipe, o grafo 25.

5.1 Teste positivo 1

Arquivo: teste_positivo_1.pl

Este grafo possui 5 vértices. Os vértices a, b, c e d formam uma clique de tamanho 4, pois todos os pares entre eles são adjacentes. O vértice e aparece como vértice extra ligado a a, mas não interfere na existência da clique.

Resultado esperado:

```
clique4.  
true.
```

E, usando `mostra_clique4`:

```
Clique de tamanho 4 encontrada: {a, b, c, d}
```

5.2 Teste positivo 2

Arquivo: teste_positivo_2.pl

Este grafo possui 7 vértices. A clique de tamanho 4 é formada pelos vértices 2, 3, 4 e 5, pois existem arestas entre todos os pares desse conjunto.

Resultado esperado:

```
clique4.  
true.
```

E, usando `mostra_clique4`:

```
Clique de tamanho 4 encontrada: {2, 3, 4, 5}
```

5.3 Teste negativo 1

Arquivo: teste_negativo_1.pl

Este grafo possui 6 vértices e é formado por dois triângulos ligados por uma ponte:

- triângulo $\{a,b,c\}$;
- triângulo $\{d,e,f\}$;
- aresta de ligação entre c e d.

Apesar de existirem cliques de tamanho 3, não existe nenhum conjunto de quatro vértices em que todos os pares sejam adjacentes. Portanto, o grafo não possui clique de tamanho 4.

Resultado esperado:

```
clique4.  
false.
```

E, usando mostra_clique4:

```
Nenhuma clique de tamanho 4 encontrada.
```

5.4 Teste negativo 2

Arquivo: teste_negativo_2.pl

Este grafo possui 8 vértices e representa um grafo bipartido completo $K_{4,4}$, com os conjuntos $\{a,b,c,d\}$ e $\{1,2,3,4\}$. Em um grafo bipartido, não existem arestas entre vértices do mesmo conjunto. Por isso, não há triângulos e também não pode existir clique de tamanho 4.

Resultado esperado:

```
clique4.  
false.
```

E, usando mostra_clique4:

```
Nenhuma clique de tamanho 4 encontrada.
```

5.5 Teste negativo 3 (caso-limite)

Arquivo: teste_negativo_3.pl

Este grafo possui 5 vértices e é quase idêntico ao Teste Positivo 1, mas remove a aresta entre os vértices c e d. Assim, o conjunto $\{a, b, c, d\}$ possui 5 das 6 adjacências necessárias para formar uma clique de tamanho 4, faltando apenas a aresta c-d. Esse caso testa o limite exato da definição de clique, comprovando que o predicado exige a existência de todas as 6 adjacências, sem aceitar quase-cliques.

Resultado esperado:

```
clique4.  
false.
```

E, usando mostra_clique4:

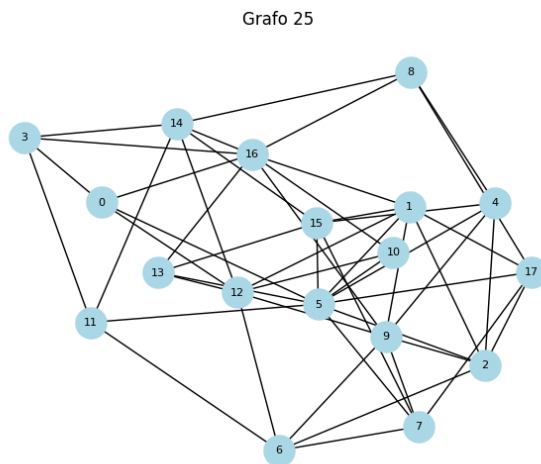
```
Nenhuma clique de tamanho 4 encontrada.
```

5.6 Grafo 25

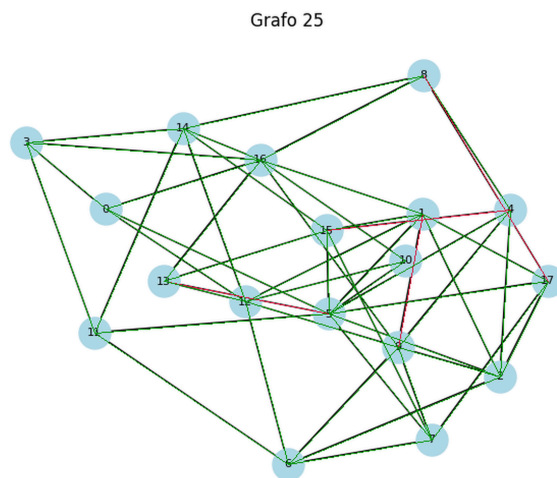
O grafo escolhido foi o Grafo 25, composto por 18 vértices (enumeradas de 0 a 17) e 50 arestas. Assim como nos demais casos de teste, ele foi representado em Prolog por meio de

**UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE RUSSAS**

fatos $\text{adj}(X, Y)$, com cada aresta declarada nas duas direções, já que o grafo é não direcionado. A figura abaixo mostra o grafo 25:



Como este grafo possui muitas arestas, algumas ligações ficavam parcialmente encobertas pelos vértices, dificultando a visualização. Por isso, elaboramos também uma segunda versão da imagem, com as arestas mais destacadas, tornando todas as ligações mais visíveis:



O grafo 25 foi o grafo grande escolhido pela equipe. Ele foi representado em Prolog por meio de fatos $\text{adj}(X,Y)$ em ambas as direções para cada aresta. Durante os testes, o predicado encontrou uma clique de tamanho 4 no grafo 25, sendo ela:

$\{1, 2, 5, 17\}$

Isso ocorre porque existem as adjacências:

$\text{adj}(1,2) . \text{adj}(1,5)$

$\text{adj}(1,17) . \text{adj}(2,5)$

$\text{adj}(2,17) . \text{adj}(5,17)$

Resultado esperado:

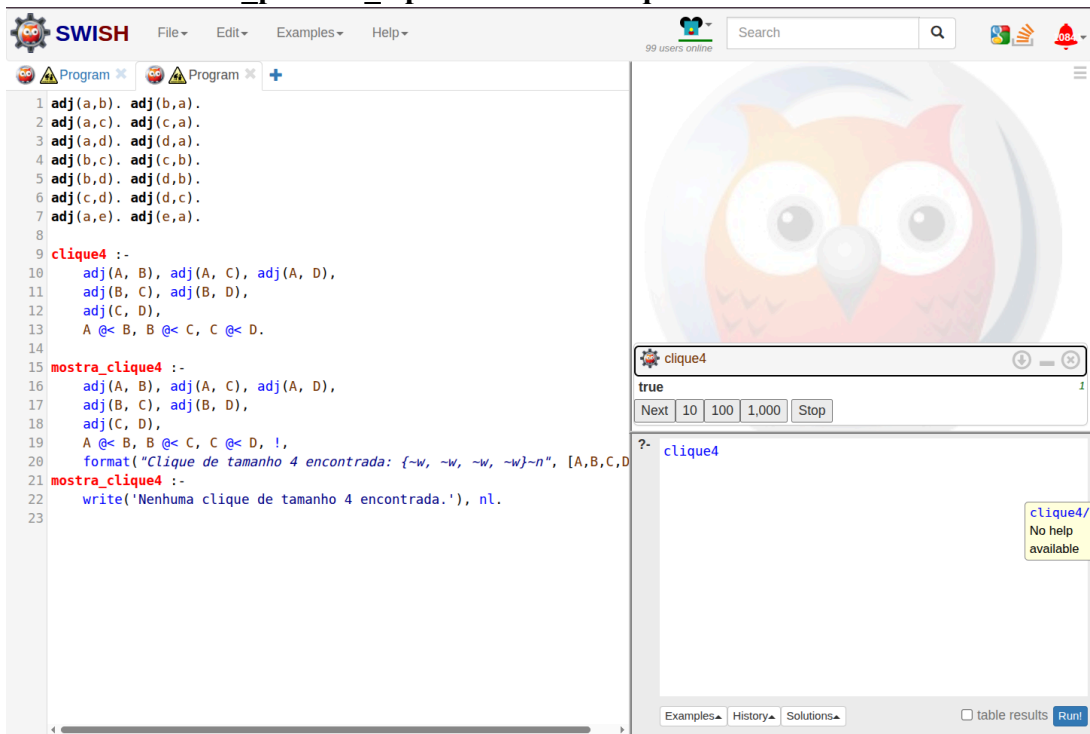
UNIVERSIDADE FEDERAL DO CEARÁ CAMPUS DE RUSSAS

```
clique4.  
true.
```

6. Capturas de tela das consultas

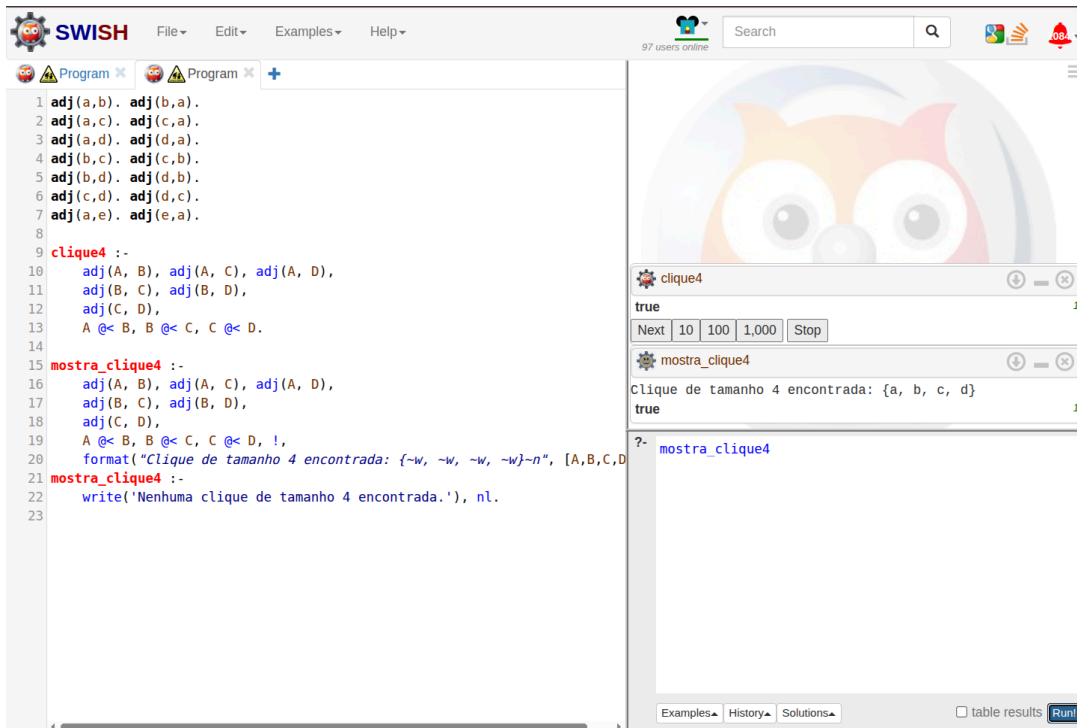
Nesta seção devem ser inseridas as capturas de tela do SWI-Prolog/SWISH mostrando a execução das consultas.

6.1 Print do teste_positivo_1.pl executando clique4.



6.2 Print do teste_positivo_1.pl executando mostra_clique4.

UNIVERSIDADE FEDERAL DO CEARÁ CAMPUS DE RUSSAS



The screenshot shows the SWISH web interface with a Prolog program loaded. The program defines adjacency relations and a predicate to find cliques of size 4. The execution results on the right show that a clique of size 4 was found, containing the elements {a, b, c, d}.

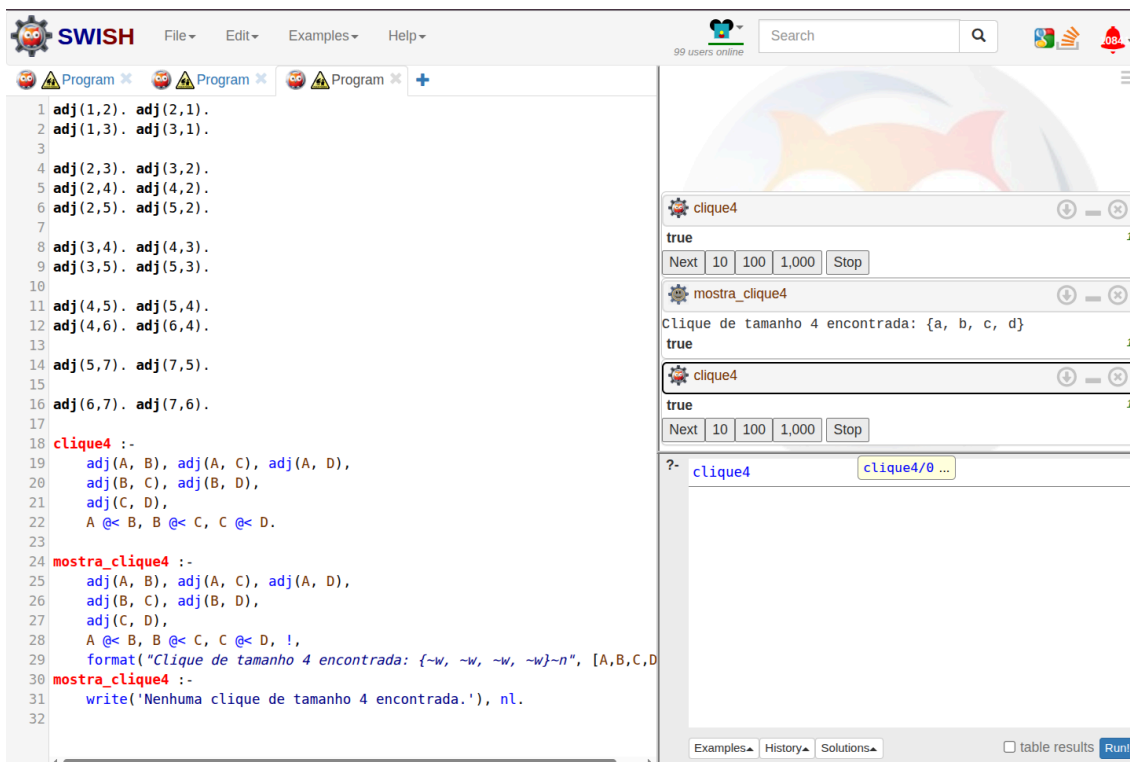
```
1 adj(a,b). adj(b,a).
2 adj(a,c). adj(c,a).
3 adj(a,d). adj(d,a).
4 adj(b,c). adj(c,b).
5 adj(b,d). adj(d,b).
6 adj(c,d). adj(d,c).
7 adj(a,e). adj(e,a).
8
9 clique4 :-
10     adj(A,B), adj(A,C), adj(A,D),
11     adj(B,C), adj(B,D),
12     adj(C,D),
13     A @< B, B @< C, C @< D.
14
15 mostra_clique4 :-
16     adj(A,B), adj(A,C), adj(A,D),
17     adj(B,C), adj(B,D),
18     adj(C,D),
19     A @< B, B @< C, C @< D, !,
20     format("Clique de tamanho 4 encontrada: {~w, ~w, ~w, ~w}-n", [A,B,C,D]),
21     mostra_clique4 :-
22     write('Nenhuma clique de tamanho 4 encontrada.'). nl.
23
```

Execution results:

```
clique4
true
Next 10 100 1,000 Stop

mostra_clique4
Clique de tamanho 4 encontrada: {a, b, c, d}
true
```

6.3 Print do teste_positivo_2.pl executando clique4.



The screenshot shows the SWISH web interface with a Prolog program loaded. The program defines adjacency relations and a predicate to find cliques of size 4. The execution results on the right show that a clique of size 4 was found, containing the elements {a, b, c, d}.

```
1 adj(1,2). adj(2,1).
2 adj(1,3). adj(3,1).
3
4 adj(2,3). adj(3,2).
5 adj(2,4). adj(4,2).
6 adj(2,5). adj(5,2).
7
8 adj(3,4). adj(4,3).
9 adj(3,5). adj(5,3).
10
11 adj(4,5). adj(5,4).
12 adj(4,6). adj(6,4).
13
14 adj(5,7). adj(7,5).
15
16 adj(6,7). adj(7,6).
17
18 clique4 :-
19     adj(A,B), adj(A,C), adj(A,D),
20     adj(B,C), adj(B,D),
21     adj(C,D),
22     A @< B, B @< C, C @< D.
23
24 mostra_clique4 :-
25     adj(A,B), adj(A,C), adj(A,D),
26     adj(B,C), adj(B,D),
27     adj(C,D),
28     A @< B, B @< C, C @< D, !,
29     format("Clique de tamanho 4 encontrada: {~w, ~w, ~w, ~w}-n", [A,B,C,D]),
30     mostra_clique4 :-
31     write('Nenhuma clique de tamanho 4 encontrada.'). nl.
32
```

Execution results:

```
clique4
true
Next 10 100 1,000 Stop

mostra_clique4
Clique de tamanho 4 encontrada: {a, b, c, d}
true

clique4
clique4/0 ...
```

6.4 Print do teste_positivo_2.pl executando mostra_clique4.

UNIVERSIDADE FEDERAL DO CEARÁ

CAMPUS DE RUSSAS

The screenshot shows the SWISH online programming environment. The left pane contains a Prolog program with the following code:

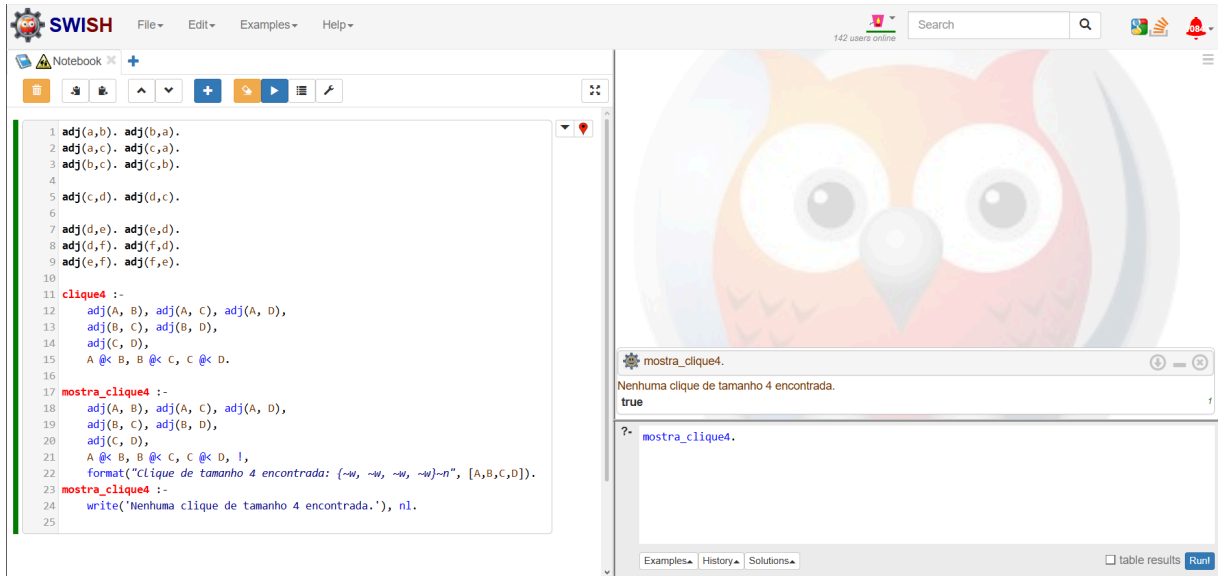
```

1 adj(1,2). adj(2,1).
2 adj(1,3). adj(3,1).
3
4 adj(2,3). adj(3,2).
5 adj(2,4). adj(4,2).
6 adj(2,5). adj(5,2).
7
8 adj(3,4). adj(4,3).
9 adj(3,5). adj(5,3).
10
11 adj(4,5). adj(5,4).
12 adj(4,6). adj(6,4).
13
14 adj(5,7). adj(7,5).
15
16 adj(6,7). adj(7,6).
17
18 clique4 :-
19     adj(A, B), adj(A, C), adj(A, D),
20     adj(B, C), adj(B, D),
21     adj(C, D),
22     A @< B, B @< C, C @< D.
23
24 mostra_clique4 :-
25     adj(A, B), adj(A, C), adj(A, D),
26     adj(B, C), adj(B, D),
27     adj(C, D),
28     A @< B, B @< C, C @< D, !,
29     format("Clique de tamanho 4 encontrada: {~w, ~w, ~w, ~w}~n", [A,B,C,D],
30     mostra_clique4 :-
31     write('Nenhuma clique de tamanho 4 encontrada.'). nl.
32
  
```

The right pane shows the execution results for the `clique4` and `mostra_clique4` predicates. The `clique4` predicate is shown twice, both times returning `true`. The `mostra_clique4` predicate is shown twice, both times displaying the message "Clique de tamanho 4 encontrada: {a, b, c, d}" and "Clique de tamanho 4 encontrada: {2, 3, 4, 5}" respectively. The bottom of the right pane shows the `mostra_clique4` predicate being called with the query `?- mostra_clique4`.

UNIVERSIDADE FEDERAL DO CEARÁ CAMPUS DE RUSSAS

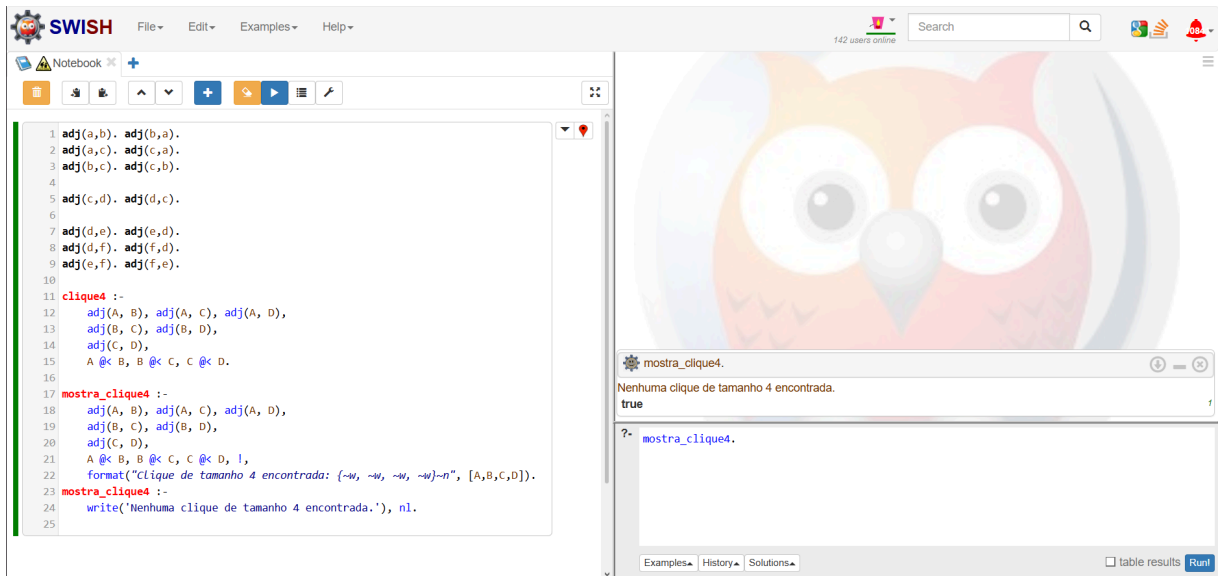
6.5 Print do teste_negativo_1.pl executando clique4.



```
1 adj(a,b). adj(b,a).
2 adj(a,c). adj(c,a).
3 adj(b,c). adj(c,b).
4
5 adj(c,d). adj(d,c).
6
7 adj(d,e). adj(e,d).
8 adj(d,f). adj(f,d).
9 adj(e,f). adj(f,e).
10
11 clique4 :-
12     adj(A, B), adj(A, C), adj(A, D),
13     adj(B, C), adj(B, D),
14     adj(C, D),
15     A @< B, B @< C, C @< D.
16
17 mostra_clique4 :-
18     adj(A, B), adj(A, C), adj(A, D),
19     adj(B, C), adj(B, D),
20     adj(C, D),
21     A @< B, B @< C, C @< D, !,
22     format("Clique de tamanho 4 encontrada: {~w, ~w, ~w, ~w}-n", [A,B,C,D]).
23 mostra_clique4 :-
24     write('Nenhuma clique de tamanho 4 encontrada.'), nl.
25
```

mostra_clique4.
Nenhuma clique de tamanho 4 encontrada.
true

6.6 Print do teste_negativo_1.pl executando mostra_clique4.

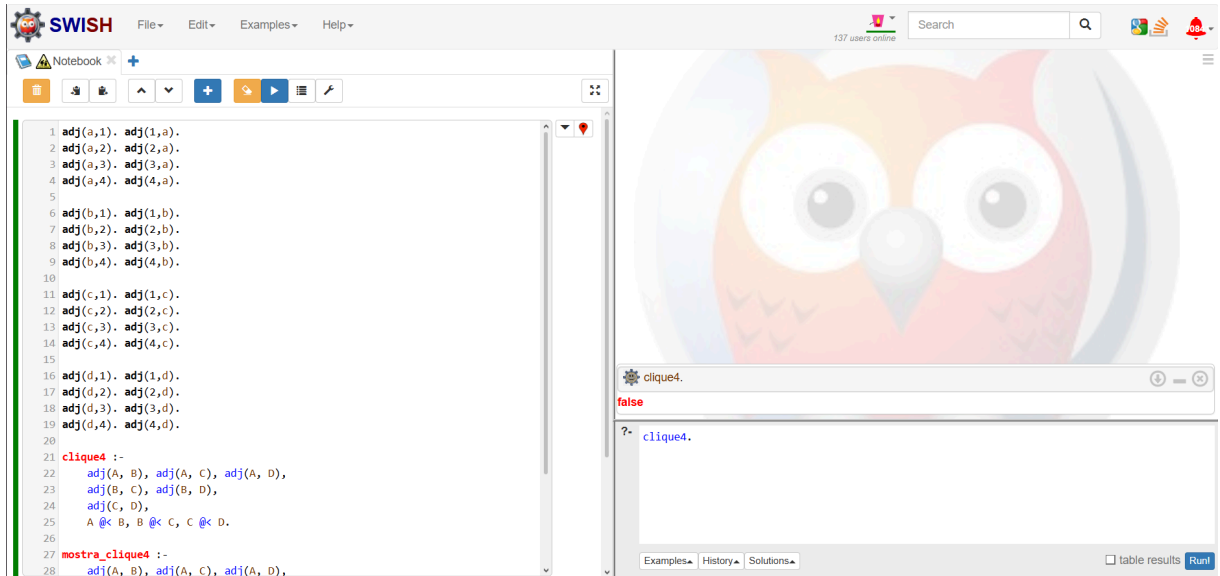


```
1 adj(a,b). adj(b,a).
2 adj(a,c). adj(c,a).
3 adj(b,c). adj(c,b).
4
5 adj(c,d). adj(d,c).
6
7 adj(d,e). adj(e,d).
8 adj(d,f). adj(f,d).
9 adj(e,f). adj(f,e).
10
11 clique4 :-
12     adj(A, B), adj(A, C), adj(A, D),
13     adj(B, C), adj(B, D),
14     adj(C, D),
15     A @< B, B @< C, C @< D.
16
17 mostra_clique4 :-
18     adj(A, B), adj(A, C), adj(A, D),
19     adj(B, C), adj(B, D),
20     adj(C, D),
21     A @< B, B @< C, C @< D, !,
22     format("Clique de tamanho 4 encontrada: {~w, ~w, ~w, ~w}-n", [A,B,C,D]).
23 mostra_clique4 :-
24     write('Nenhuma clique de tamanho 4 encontrada.'), nl.
25
```

mostra_clique4.
Nenhuma clique de tamanho 4 encontrada.
true

UNIVERSIDADE FEDERAL DO CEARÁ CAMPUS DE RUSSAS

6.7 Print do teste_negativo_2.pl executando clique4.

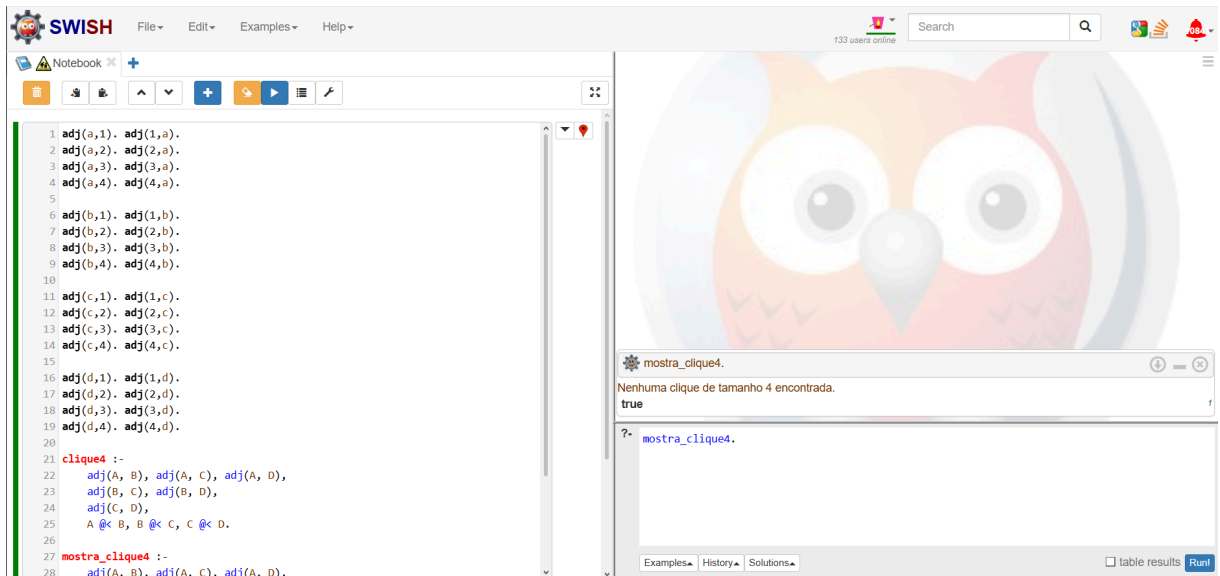


```
1 adj(a,1). adj(1,a).
2 adj(a,2). adj(2,a).
3 adj(a,3). adj(3,a).
4 adj(a,4). adj(4,a).
5
6 adj(b,1). adj(1,b).
7 adj(b,2). adj(2,b).
8 adj(b,3). adj(3,b).
9 adj(b,4). adj(4,b).
10
11 adj(c,1). adj(1,c).
12 adj(c,2). adj(2,c).
13 adj(c,3). adj(3,c).
14 adj(c,4). adj(4,c).
15
16 adj(d,1). adj(1,d).
17 adj(d,2). adj(2,d).
18 adj(d,3). adj(3,d).
19 adj(d,4). adj(4,d).
20
21 clique4 :-
22     adj(A, B), adj(A, C), adj(A, D),
23     adj(B, C), adj(B, D),
24     adj(C, D),
25     A @< B, B @< C, C @< D.
26
27 mostra_clique4 :-
28     adj(A, B), adj(A, C), adj(A, D),
```

clique4.
false

?- clique4.

6.8 Print do teste_negativo_2.pl executando mostra_clique4.



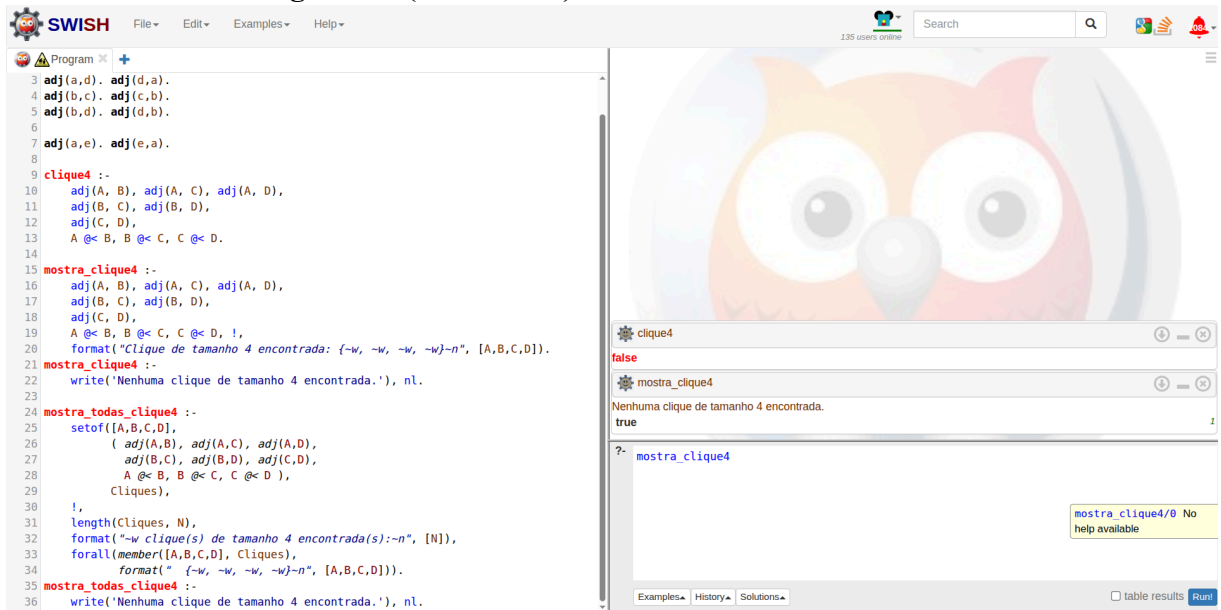
```
1 adj(a,1). adj(1,a).
2 adj(a,2). adj(2,a).
3 adj(a,3). adj(3,a).
4 adj(a,4). adj(4,a).
5
6 adj(b,1). adj(1,b).
7 adj(b,2). adj(2,b).
8 adj(b,3). adj(3,b).
9 adj(b,4). adj(4,b).
10
11 adj(c,1). adj(1,c).
12 adj(c,2). adj(2,c).
13 adj(c,3). adj(3,c).
14 adj(c,4). adj(4,c).
15
16 adj(d,1). adj(1,d).
17 adj(d,2). adj(2,d).
18 adj(d,3). adj(3,d).
19 adj(d,4). adj(4,d).
20
21 clique4 :-
22     adj(A, B), adj(A, C), adj(A, D),
23     adj(B, C), adj(B, D),
24     adj(C, D),
25     A @< B, B @< C, C @< D.
26
27 mostra_clique4 :-
28     adj(A, B), adj(A, C), adj(A, D),
```

mostra_clique4.
Nenhuma clique de tamanho 4 encontrada.
true

?- mostra_clique4.

UNIVERSIDADE FEDERAL DO CEARÁ CAMPUS DE RUSSAS

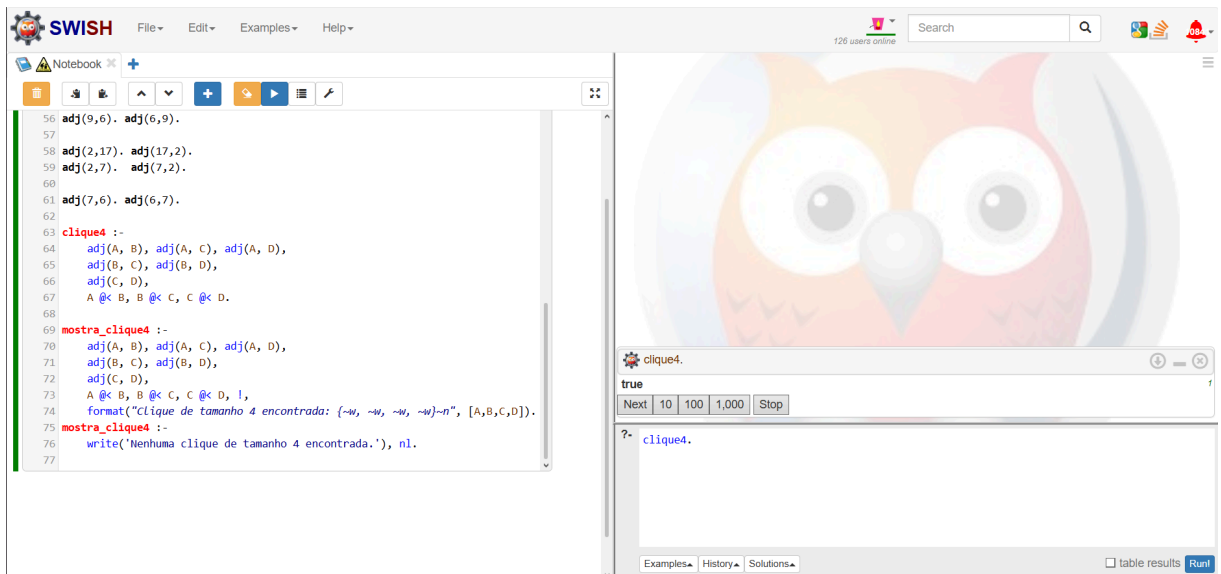
6.9 Print do Teste negativo 3 (caso-limite)



The screenshot shows the SWISH environment with a Prolog program in the left pane and its execution results in the right pane. The program defines adjacency relations and searches for cliques of size 4. The results pane shows the execution of `clique4` returning `false`, `mostra_clique4` returning `Nenhuma clique de tamanho 4 encontrada.`, and a query for `mostra_clique4` returning `mostra_clique4/0 No help available`.

```
3 adj(a,d). adj(d,a).
4 adj(b,c). adj(c,b).
5 adj(b,d). adj(d,b).
6
7 adj(a,e). adj(e,a).
8
9 clique4 :-
10   adj(A, B), adj(A, C), adj(A, D),
11   adj(B, C), adj(B, D),
12   adj(C, D),
13   A @< B, B @< C, C @< D.
14
15 mostra_clique4 :-
16   adj(A, B), adj(A, C), adj(A, D),
17   adj(B, C), adj(B, D),
18   adj(C, D),
19   A @< B, B @< C, C @< D, !,
20   format("Clique de tamanho 4 encontrada: {-w, -w, -w, -w}-n", [A,B,C,D]).
21 mostra_clique4 :-
22   write('Nenhuma clique de tamanho 4 encontrada.'). nl.
23
24 mostra_todas_clique4 :-
25   setof([A,B,C,D],
26     ( adj(A,B), adj(A,C), adj(A,D),
27       adj(B,C), adj(B,D), adj(C,D),
28       A @< B, B @< C, C @< D ),
29     Cliques),
30   !,
31   length(Cliques, N),
32   format("~w clique(s) de tamanho 4 encontrada(s):-n", [N]),
33   forall(member([A,B,C,D], Cliques),
34     format(" {-w, -w, -w, -w}-n", [A,B,C,D])).
35 mostra_todas_clique4 :-
36   write('Nenhuma clique de tamanho 4 encontrada.'). nl.
```

6.10 Print do grafo 25 executando clique4.

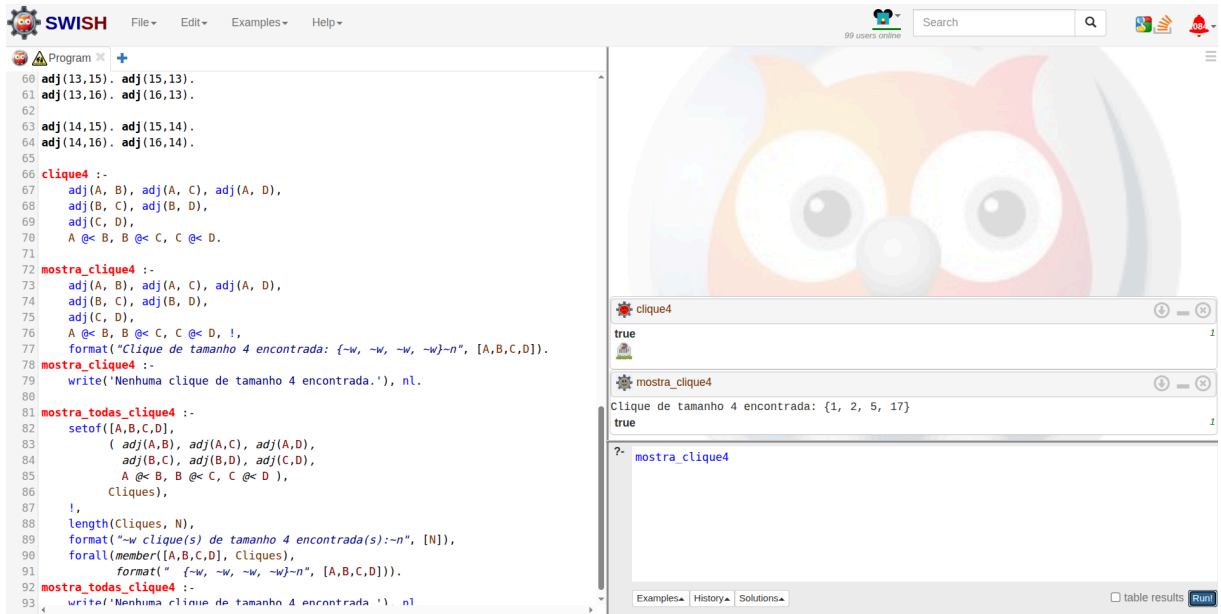


The screenshot shows the SWISH environment with a Prolog program in the left pane and its execution results in the right pane. The program defines adjacency relations and searches for cliques of size 4. The results pane shows the execution of `clique4` returning `true`, and a query for `clique4` returning `clique4.`.

```
56 adj(9,6). adj(6,9).
57
58 adj(2,17). adj(17,2).
59 adj(2,7). adj(7,2).
60
61 adj(7,6). adj(6,7).
62
63 clique4 :-
64   adj(A, B), adj(A, C), adj(A, D),
65   adj(B, C), adj(B, D),
66   adj(C, D),
67   A @< B, B @< C, C @< D.
68
69 mostra_clique4 :-
70   adj(A, B), adj(A, C), adj(A, D),
71   adj(B, C), adj(B, D),
72   adj(C, D),
73   A @< B, B @< C, C @< D, !,
74   format("Clique de tamanho 4 encontrada: {-w, -w, -w, -w}-n", [A,B,C,D]).
75 mostra_clique4 :-
76   write('Nenhuma clique de tamanho 4 encontrada.'). nl.
77
```

6.11 Print do grafo 25 executando mostra_clique4.

UNIVERSIDADE FEDERAL DO CEARÁ CAMPUS DE RUSSAS



The screenshot shows the SWISH Prolog IDE interface. On the left, a code editor displays a Prolog program with the following lines:

```
60 adj(13,15). adj(15,13).
61 adj(13,16). adj(16,13).
62
63 adj(14,15). adj(15,14).
64 adj(14,16). adj(16,14).
65
66 clique4 :-
67     adj(A, B), adj(A, C), adj(A, D),
68     adj(B, C), adj(B, D),
69     adj(C, D),
70     A @< B, B @< C, C @< D.
71
72 mostra_clique4 :-
73     adj(A, B), adj(A, C), adj(A, D),
74     adj(B, C), adj(B, D),
75     adj(C, D),
76     A @< B, B @< C, C @< D, !,
77     format("Clique de tamanho 4 encontrada: {-w, ~w, ~w, ~w}~n", [A,B,C,D]).
78 mostra_clique4 :-
79     write('Nenhuma clique de tamanho 4 encontrada.'), nl.
80
81 mostra_todas_clique4 :-
82     setof([A,B,C,D],
83         ( adj(A,B), adj(A,C), adj(A,D),
84           adj(B,C), adj(B,D), adj(C,D),
85           A @< B, B @< C, C @< D ),
86         Cliques),
87     !,
88     length(Cliques, N),
89     format("-w clique(s) de tamanho 4 encontrada(s):~n", [N]),
90     forall(member([A,B,C,D], Cliques),
91         format(" {-w, ~w, ~w, ~w}~n", [A,B,C,D])).
92 mostra_todas_clique4 :-
93     write('Nenhuma clique de tamanho 4 encontrada.'), nl.
```

On the right, the execution results are shown in a window titled "clique4". The results are:

```
true
mostra_clique4
Clique de tamanho 4 encontrada: {1, 2, 5, 17}
true
?- mostra_clique4
```

7. Resultados obtidos

Os testes positivos retornaram sucesso para o predicado clique4, confirmando que a implementação consegue identificar grafos que possuem clique de tamanho 4. Os testes negativos retornaram falha para clique4, confirmando que a implementação não indica a existência de clique quando a propriedade não é satisfeita.

No grafo 25, o predicado também retornou sucesso, pois foi encontrada pelo menos uma clique de tamanho 4. Um exemplo identificado foi {1, 2, 5, 17}.

UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE RUSSAS

Resumo dos resultados:

Arquivo	Quantidade de vértices	Tipo de teste	Resultado esperado
teste_positivo_1.pl	5	Positivo	Possui clique de tamanho 4
teste_positivo_2.pl	7	Positivo	Possui clique de tamanho 4
teste_negativo_1.pl	6	Negativo	Não possui clique de tamanho 4
teste_negativo_2.pl	8	Negativo	Não possui clique de tamanho 4
grafo25.pl	18	Grafo grande	Possui clique de tamanho 4

8. Conclusão

A implementação em SWI-Prolog foi capaz de resolver o problema de detecção de clique de tamanho 4 por meio da verificação das adjacências entre todos os pares de quatro vértices.

O uso do backtracking permitiu que o Prolog testasse diferentes combinações de vértices automaticamente. A ordenação com `@<` evitou repetições da mesma clique em ordens diferentes, tornando a busca mais organizada.

Os testes realizados demonstraram que o predicado funciona corretamente tanto para casos em que a clique existe quanto para casos em que ela não existe. O teste final com o grafo 25 também confirmou a existência de uma clique de tamanho 4 no grafo escolhido pela equipe.